Incognito

Anonymizing Qualitative Research Data Locally
in the browser — detection, review, and person graphs

CamemBERT · GLiNER · Albert API · UDPipe
— client-side pipelines for the humanities
and social sciences.

Xiaoou Wang · Digital Humanities Engineer
MSHS Sud-Est · Université Côte d'Azur
Incognito Web 1.0.0 · AGPLv3 · ~20 minutes

Agenda (20 minutes)

	Section	~min
1	Problem & approach	3
2	Web architecture (client-side)	2
3	Pipeline **Basic** (CamemBERT)	3
4	Pipeline **Advanced** (GLiNER)	3
5	Pipeline **Generative** (Albert API)	2.5
6	Person graph	3
7	Limitations & future directions	1.5

Key idea: each mode has a different pipeline, but all share the same human review layer.

1 · Problem & approach

Anonymize without losing the research context

- Interviews / field notes: names, places, employers, health, qualifications…
- Risk of **re-identification** through combinations of details
- Trade-off: protect participants **while** keeping the text analyzable
- Commercial solutions may work — but require **sending the corpus off-device**

> Incognito takes a different approach: **local processing** + **human oversight** + **traceability**.

What Incognito offers

Detect → review → replace → export

[PER_1]
[LOC_1]
[ORG_1]
[EMA_1]
[NAT_1]

· 3-letter category code + stable index

Principle	Web implementation
Local-first	ONNX / WASM in the browser (except Albert)
Human in the loop	Soft-exclude, spans manuels, Related to
Traceability	Audit Markdown, CSV graphe
Multiple pipelines	3 engines, 1 review layer

An anonymization assistant — not a guarantee of complete anonymity.

2 · Web architecture

Stack client-side

```
Browser interface  →  background detection (without freezing the UI)  →  review, person graph, exports
```

Layer	Tech · Framework	Role
UI	React (Vite), FR/EN	3+1 review layout, batch processing, progress
Basic NER	Transformers.js + ONNX Runtime WASM	Token-classification CamemBERT / BERT
Advanced	`gliner` + ONNX `q4f16` (~472 Mo)	Open labels, chunking
Generative	`fetch` → Worker CORS → Albert `/v1`	Compact graph JSON
Graphe	UDPipe WASM French-GSD (~23 Mo)	Dependencies + proximity-based merging
Hosting	GitHub Pages (statique) + Cloud EOSC	

Where does the data go?

Basic & Advanced

Text **stays** in the browser
One-time download of HF model weights
then **browser caching**

Generative

Text + API key → **Albert**
via Worker EOSC (European Open Science Cloud)

`Pages github.io → Worker /v1/* → albert.api.etalab.gouv.fr`

3 · Basic Pipeline

CamemBERT / Transformers.js

Basic — processing pipeline

Start with pasted or imported text
→ the model identifies names, places, organizations, dates… *without blocking the UI*
→ **all occurrences** of the same value are highlighted (not just the first)
→ emails, URLs, dates, and phone numbers can also be detected with simple rules
→ results are organized by **category** (person, location…)
→ values are replaced with codes such as `[PER_1]` and a preview is displayed
→ in parallel, a **person graph** is built (with human review)

Typical models: CamemBERT + dates (default), base CamemBERT, English BERT

4 · Advanced Pipeline

Open-label GLiNER

Advanced — processing pipeline

Choose the entity types to detect (person, disease, qualification, nationality…)
→ the GLiNER model is downloaded once and then cached in the browser
→ for emails, URLs, and phone numbers, **explicit rules** are preferred over model predictions
→ results populate the on-screen categories *and* the person graph attributes
→ then the same review workflow as Basic (exclude, link, export)

Advanced — why GLiNER?

CamemBERT (Basic)	GLiNER (Advanced)
Fixed NER entity types	**Free-text** labels / configurable protocol
Good default for French interviews	Health, qualifications, job title, nationality…
More stable for PER/LOC/ORG	More flexible, sometimes noisier

- Low threshold (0.2) → high recall → **human review filters results**
- Same graph layer, with richer attributes (address, occupation, school…)

Fichiers : `glinerRuntime.js` · `useGlinerWorker.js` · `ruleDetection.js` · `GlinerWorkflowSection.jsx`

5 · Generative pipeline

Albert API

Generative — processing pipeline

First, candidate entities are identified locally (names, contact details…)
→ the text is sent to **Albert** (using the user's API key)
→ Albert returns a **compact representation**: identities, aliases, attributes, relationships → the browser aligns this representation with spans in the text
→ the anonymized preview is generated locally
→ the user can exclude, link ('Related to'), or focus on a person
→ each correction regenerates the preview

Generative — network and token constraints

- **CORS** : Albert does not properly handle preflight requests from `github.io` → allowlisted Worker proxy
- **Proxy** : forward `Authorization` + body ; **no API keys stored** in the cloud
- **max_tokens** / JSON compact : reduce `finish_reason=length`

Fichiers : `albertClient.js` · `albertCandidates.js` · `albertEdit.js` · `albert-proxy/`

6 · Graph & review

Common layer to all pipelines

Building the person graph

Start with detected entities (Basic / Advanced) or Albert’s representation
→ French dependency parsing in the browser
→ each mention is linked to its position in the text
→ information referring to the **same person** is grouped using:
  · grammatical relationships within a sentence
  · proximity within a paragraph
→ the user can correct links with **Related to** (these choices are preserved)
→ excluding a value in the categories or graph has the **same effect** on the preview

Interaction semantics

Action	Highlighted UI	Anonymized text
Person chip	Focus / underline	**No effect**
Soft-exclude (× / menu)	Greyed out	**Excluded** from replacement
Related to	Relink attributes / aliases	Rebuild + replacements

Layout 3+1 : Categories · Text · Graph — **full-width anonymized preview below**

Exports : CSV long (1 row per surface form) · CSV by person · Audit log

8 · Limitations & future directions

Technical limitations

- Quasi-identifiers / narrative context beyond NER
- **Incomplete** graph
- GLiNER / CamemBERT : entity boundaries, false positives / false negatives
- WASM : performance depends on the user’s device

> Always **review the output** before sharing or archiving.

Future directions

- Use generative models to detect complex quasi-identifiers, especially those that cannot be identified through named entity recognition alone.
- Combine qualitative analysis with quantitative anonymization methods, using the person–information graph to measure re-identification risk (k-anonymity, profile uniqueness, combinations of attributes, etc.).
- Evaluate anonymization robustness through re-identification attacks, cross-referencing public sources (INSEE, Wikidata, directories, and other open databases) to estimate real-world re-identification risk.

Takeaways

1. **Three distinct pipelines** distincts (CamemBERT · GLiNER · Albert)
2. **One shared review layer**
3. **Local-first** in the browser + Generative mode

Links

- App : xiaoouwang.github.io/Incognito
- Code : github.com/xiaoouwang/Incognito
- Development log : `docs/WEB_INTERFACE_EVOLUTION.md`

Thank you

Questions?

Xiaoou Wang · MSHS Sud-Est · Université Côte d'Azur
xiaoouwang.github.io/Incognito

AGPLv3 · Incognito Web 1.0.0